Terrain
Design Process Book

Sara Javan · 2026 · A record of decisions, friction, and teaching moments from building Terrain — an empowerment platform for 18-year-olds entering trades.

WHAT THIS IS

Each entry documents one concept from the agentic design curriculum, one real decision made while building Terrain, what broke, and what a future student needs to know. The build is the evidence. The friction is the teaching.

HOW TO READ IT

The Concept	What the books taught — the skill or paradigm.
The Decision	How it was applied in Terrain — the actual build moment.
The Friction	What broke, what surprised, what the books didn't prepare for.
The Teaching	What to tell a student before they try this themselves.

ENTRIES

01	The design-code gap	*CC4D Ch 1 + TAD Ch 1*
02	The first prototype loop	*CC4D Ch 3*
03	Metaphoric prompting	*CC4D Ch 4*
04	CLAUDE.md as design memory	*CC4D Ch 5*
05	Context drift	*CC4D Ch 4*
06	Prototype in HTML first	*CC4D Ch 3*

07	One emotion governs everything	*Design principle*
08	Horizon Reveal — sketch as brief	*TAD Ch 1*
09	The text is the design	*TAD Ch 3*
10	Token enforcement	*CC4D Ch 6 — upcoming*
11	The agent waits for conviction	*Meta-insight*
12	Prototype to product	*Architecture decision*

01 The design-code gap

Why the g

THE CONCEPT

The design artifact and the production artifact have never been the same thing. A Figma file is not a website. Every handoff introduces translation loss that accumulates — the 12px gap that ships as 16px, the hover state the engineer improved, the responsive behaviour nobody anticipated. The handoff tax isn't a people problem. It's a format problem. What AI agents change is not the gap itself — it's who can cross it, and how fast. TAD's sharpest formulation: the GUI becomes the review surface, not the authoring surface.

THE DECISION

Before touching Terrain's code, the full portfolio-2026 architecture was audited in a single Claude Code session. It read 18 files in under 2 minutes and produced a complete mental model of the codebase — components, routes, design tokens, content structure. That audit replaced what would have taken a developer half a day. The decision: build on the existing architecture, not from scratch. The gap was crossed before a single line was written.

THE FRICTION

The first attempt pointed Claude Code at the wrong project. portfolio-2026 was found but terrain was the real target. Two sessions wasted before the correct directory was located. The lesson: Claude Code's context is exactly where you launch it from. Always navigate before you prompt.

THE TEACHING

The bottleneck moves from production to decision-making. Your design judgment, your risk radar, your ability to describe visual intent — those become the scarce resource, not the ability to write CSS. The designers who struggle with agentic tools are the ones who don't yet trust their own design conviction.

"You are not learning to code. You are learning to direct."

02 The first prototype loop

THE CONCEPT

The design loop described in CC4D Ch 3: Describe intent → Claude generates code → Review in browser → Describe changes → Repeat. Three types of iteration: value adjustments (spacing, sizing, exact CSS values), feature additions (new behaviour described in design language), and structural changes (layout reflow, often with a reference screenshot). The first output is a starting point, not the finish line. Treat it like a first draft from a junior developer — the value is in the iteration and design judgment, not the initial generation.

THE DECISION

Terrain's Screen 1 landing page was built in three rounds. Round 1: gradient background, heading, subline, CTA. Round 2: warm directional gradient — described as 'like early morning light breaking through.' Round 3: cobalt gradient applied to the word 'terrain' via CSS background-clip text. Each round focused on one type of change. The screen that exists now is three rounds of design judgment applied to one initial generation.

THE FRICTION

The /clear command matters more than expected. After several rounds of feedback on the same element, context accumulates and Claude Code starts hedging on overrides. Running /clear creates a fresh session with better information — like a new artboard. Use it before the third correction on the same issue.

THE TEACHING

Sketch before you prompt. Screen 2 worked almost perfectly in round 1 because the notebook sketch was done first. The sketch was the brief. When design intent is fully formed before the session opens, the loop collapses. When it isn't, you iterate more. The prompt is a translation — and you can only translate something you already understand.

"The loop converges fastest when the design exists in your head before the session opens."

03 Metaphoric prompting

THE CONCEPT

CC4D Ch 4 establishes a specificity spectrum: vague prompts get Claude's default aesthetic (soft backgrounds, warm accents, rounded cards), specific prompts get your intent. But there's a third mode the book doesn't fully name: metaphoric prompts. Describing a feeling or a reference — 'like early morning light breaking through,' 'like a field manual,' 'like a receipt' — produces results closer to design intent than hex codes, because Claude was trained on human language, not just technical specifications.

THE DECISION

Every Terrain screen was prompted in metaphoric language first, technical values second. 'Hopeful' governed Screen 1. 'Like a worn document' governed Screen 3. 'Like sitting across from someone who actually wants to know your answer' governed Screen 4. The technical values — #121314, #241E14, #0047FF — came after the feeling was established, not before.

THE FRICTION

Metaphoric prompts work for mood and texture. They fail for precision. 'Make the button feel hopeful' produced a good gradient direction but the exact colours needed a second pass with values. The pattern: establish feeling first, specify values in round 2.

THE TEACHING

Designers who think in metaphor have a structural advantage in the prompting loop that nobody in the developer-first Claude Code world is talking about. Your years of describing design intent to developers and clients — 'it should feel premium without being cold' — is exactly the skill that makes the loop fast. Don't translate that instinct into hex codes. Use it directly.

"'Like early morning light breaking through' got further in one prompt than a hex code would have."

04 CLAUDE.md as design memory

THE CONCEPT

CLAUDE.md is the project's persistent memory. Every session in the project reads it first. It holds the design system, the voice, the principles, the architecture map, the non-negotiables. Without it, every session starts from zero and context drift compounds. With it, the agent starts already aligned. The key formulation: a well-configured agent outperforms a bare agent every time — not because it's smarter, but because the configuration does the work upfront.

THE DECISION

Terrain's CLAUDE.md was written after the token system was committed — not before. This was the right sequence. Encoding decisions that haven't been made yet produces a document that constrains rather than guides. The CLAUDE.md written after the token test session encoded real decisions: #0047FF as the single accent, #241E14 as the warm card surface, 0.75 as the minimum text opacity, buttons that fit text not containers. Every rule earned.

THE FRICTION

The first token rollout missed several elements because they were applied in a long session where earlier orange decisions had accumulated. CLAUDE.md would have prevented this — but only if it had existed from the start of that session. Lesson: create CLAUDE.md before the first build session, even if it's sparse. Add to it as decisions get made.

THE TEACHING

CLAUDE.md is not documentation. Documentation describes what exists. CLAUDE.md shapes what gets created next. The distinction matters because designers are trained to document after the fact. CLAUDE.md needs to exist before the first prompt. Start with three things: the colour palette, the typography rules, and one design principle. Add the rest as the project earns them.

"Define rules once. The agent applies them everywhere. You pay the configuration cost once and amortise it across every session."

05 Context drift

THE CONCEPT

Context drift happens when a long session accumulates so many instructions that Claude Code starts interpreting new instructions as additions rather than replacements. It remembers earlier decisions as intentional and hedges on overriding them. The symptom: you ask Claude Code to replace a colour and it reports zero replacements found, even though the colour is clearly visible in the browser. The root cause: earlier in the session, that colour was explicitly requested. The agent treats the override as a contradiction rather than a correction.

THE DECISION

During Terrain's token rollout, orange (#FF6B00) was specified early as an accent option. Later it was fully committed out in favour of cobalt (#0047FF). Despite multiple instructions to remove it, fragments persisted across three separate searches. The fix required explicitly naming the override: 'Search the entire file for any hex values in the orange/amber range. List every instance found with its line number. Replace each one. Confirm the exact count replaced and confirm zero instances remain.'

THE FRICTION

The word 'confirm' is doing more work than it looks like. Asking Claude Code to confirm a count forces it to actually perform a full search rather than assuming the previous partial search was complete. Without the confirmation request, it's possible to get a 'done' response that isn't.

THE TEACHING

Two-shot resolution for context drift: Shot 1 — audit before change ('search the entire file for X, list every instance with line number'). Shot 2 — override with count confirmation ('replace every instance, confirm the exact number replaced, confirm zero remain'). The CLAUDE.md is the structural fix — strong design memory prevents drift from accumulating in the first place.

"Context drift is a long-session problem. Fresh sessions with strong CLAUDE.md don't drift. The rules are encoded at the start, not accumulated through conversation."

06 Prototype in HTML first

THE CONCEPT

CC4D Ch 3 uses plain HTML with inline CSS deliberately — no frameworks, no build tools. The goal is to learn the design loop, not a framework. Single-file HTML is the fastest path from idea to browser. The principle generalises: any format that reduces friction between idea and visible output is the right format for the discovery phase. Commit to architecture only after the design decisions are stable.

THE DECISION

Terrain's four screens were built as plain HTML files in a prototype/ folder inside portfolio-2026. All design decisions — token system, typography, layout, interactions, colour — were made and validated in HTML. Only when the token system was fully committed did the project migrate to a standalone Next.js application. The HTML files were preserved in a /prototype subfolder as permanent reference.

THE FRICTION

The migration revealed that the HTML files were unreachable via the Next.js dev server. Files in subdirectories don't route through the Next.js app router. Lesson: prototype files and production files need to be kept deliberately separate from the start, not just in different folders but in different mental categories.

THE TEACHING

Committing to Next.js before the token system was settled would have meant rebuilding CSS architecture twice — once in HTML as exploration, once in Tailwind as production. HTML first means you build once with confidence. The framework is not the product. The design decisions are the product. The framework is just how they ship.

"Prototype in HTML. Graduate to a framework when decisions are committed. Build once."

07 One emotion governs everything

THE CONCEPT

Every design system has explicit rules — tokens, typography, spacing. The rules govern the how. But strong products also have an implicit rule — one emotional truth that governs the why. When decisions are ambiguous, the emotional truth resolves them. It's not a brand value or a mission statement. It's a specific, testable feeling that either a design element produces or it doesn't.

THE DECISION

Before the first line of Terrain's code was written, one word was named: hopeful. Not inspired (too big and too distant), not confident (too transactional). Hopeful. The feeling that the future is navigable. That the path exists even if you can't see all of it yet. Every design decision since has been tested against it. When the flat dark background didn't feel hopeful, it became a gradient. When the original button felt like a transactional click, it became a warm glow. The word 'hopeful' resolved both decisions without requiring a lengthy specification.

THE FRICTION

The risk of a single governing emotion is that it can be used to justify almost anything. 'This feels hopeful to me' is not a testable prompt instruction. The emotion needs to be translated into specific design language before it's useful in a session — warmth, directionality, light moving through darkness, something worth reaching for. The word is the compass. The translation is the map.

THE TEACHING

Before starting any project with Claude Code, name the one feeling the product needs to produce. Not a feature, not a user goal, not a brand value. A feeling. Then translate it into three design directions: a colour principle, a typography principle, and an interaction principle. Those three translations become the most useful part of your CLAUDE.md.

"When you're unsure about any design decision, ask one question: does this feel hopeful? If not, it doesn't belong."

08 Horizon Reveal — sketch as brief

THE CONCEPT

TAD's first chapter describes the shift from GUI as authoring surface to GUI as review surface. The agent authors. The designer reviews. But there's a third surface the book doesn't name explicitly: the notebook. Physical sketching before a Claude Code session is not a planning step. It's the prompt in its most precise form. The sketch encodes layout, hierarchy, interaction intent, and emotional tone in a language the agent can read through a screenshot.

THE DECISION

Screen 2's Horizon Reveal was sketched before any prompt was written. The sketch showed full-screen illustrated cards, swipe navigation, zero menu entry, adjacent cards peeking in from the sides, pattern name dominant at the bottom. The prompt translated the sketch directly. The screen worked almost perfectly in round 1 — not because the prompt was clever, but because the design intent was fully formed before the session opened.

THE FRICTION

Claude Code added decorative shapes inside the placeholder cards unprompted — a circle element in the top left. It was creative in the absence of complete specification. The fix was a single instruction: 'Remove all decorative elements inside cards. Pure gradient only.' The lesson: where you leave gaps, Claude Code fills them with its own aesthetic. Specify emptiness as deliberately as content.

THE TEACHING

Sketch before you prompt. Not wireframes — fast sketches. The sketch forces you to make the decisions that matter before the session opens. What's the layout? What's dominant? What's absent? Once those questions are answered on paper, the prompt writes itself. The loop converges in one or two rounds instead of five.

"The notebook sketch is not preparation. It is the brief."

09 The text is the design

When wri

THE CONCEPT

TAD Chapter 3 introduces the concept of teaching agents to design through skill files and design harnesses — structured prompts that encode not just what to build but how to make design decisions. The deeper principle: when the content itself carries enough weight, the design's job is to get out of the way. Typography, texture, and whitespace serve the writing. Not the reverse.

THE DECISION

Blake's worker story was built with no images, no illustrations, no visual elements beyond typography and a dark textured background. Five sections, vertical scroll, impact statements at display scale, a monospace receipt. The receipt for Section 04 — itemised tool costs — is the most powerful moment on the screen. Not because of design. Because of what it costs. The design decision was restraint: let the numbers sit in silence.

THE FRICTION

The first version had quotation marks around the impact statements. 'I thought electricity was the job. It wasn't.' vs I thought electricity was the job. It wasn't. The quotes made it feel cited. Removing them made it feel stated. One character change. Significant difference in authority.

THE TEACHING

Before adding a visual element, ask whether the writing is doing enough work. If the text is honest and specific, decoration competes with it. The visual dictionary — illustrated trade terminology inline with worker stories — is parked for a later build precisely because the text version needed to prove itself first. Add visuals when they earn their place, not as default.

"'Your body is the real tool.' At display scale. No quotation marks. That's the whole design."

10 Token enforcement

THE CONCEPT

CC4D Chapter 6 shows how design tokens move from CSS custom properties (advisory) to TypeScript token files imported by Tailwind (enforced). The distinction matters: CSS variables in CLAUDE.md describe what should happen. TypeScript token files imported into tailwind.config.ts make it impossible to use values outside the system without actively overriding an import. One is documentation. The other is architecture.

THE DECISION

Terrain's token system currently lives in CLAUDE.md as CSS custom properties. The next build session will move them to src/tokens/colors.ts, src/tokens/typography.ts, and src/tokens/spacing.ts — then generate a tailwind.config.ts that extends from those files. This transforms the design system from a set of guidelines the agent is asked to follow into a set of constraints the architecture enforces.

THE FRICTION

Not yet built. Entry written in advance as a teaching note for the session where it happens.

THE TEACHING

There are two kinds of design system rules: the ones you ask people to follow and the ones the system enforces. Claude Code will follow CLAUDE.md guidelines — until context drift, a long session, or an ambiguous prompt causes it to default. Token files imported into the build system cannot be defaulted. Build the enforcement layer as soon as the token decisions are stable.

"A well-structured token system is the most valuable design asset you can give an agent. More valuable than a Figma file. More valuable than a component library."

11

The agent waits for conviction

THE CONCEPT

This insight doesn't come from either book. It emerged from the first real build session with Claude Code on Terrain. It is the most important thing in this document. The agent has capability but no conviction. It can execute anything you can describe. But it will not decide what to describe. It will not choose which feeling matters. It will not know that 'hopeful' is the governing emotion or that 'terrain' is the right word or that the receipt needs to sit in silence. Those decisions are yours. The agent waits.

THE DECISION

Every Terrain design decision — the gradient direction, the cobalt word, the impact statement without quotation marks, the receipt format, the three questions on Screen 4 — came from a specific human decision about what mattered. When those decisions were clear and expressed with conviction, the agent executed them in one or two rounds. When they were vague or uncertain, the agent filled the gap with its own aesthetic defaults. The speed of the loop is a direct function of the clarity of the conviction.

THE FRICTION

The temptation is to let the agent decide when you're unsure. 'Make it look good' is a real prompt. It produces real output. But the output is the agent's aesthetic, not yours. The only way to get your design into the output is to know what your design is before you open the session.

THE TEACHING

Tell your students this before their first session: you are not learning to use a tool. You are learning to know your own design conviction well enough to express it clearly. The tool is easy. The conviction takes years. You already have it. The curriculum is teaching you to trust it.

"Claude didn't want to cross the boundaries. It was waiting for me to inject more confidence and shape it."

12 Prototype to product

THE CONCEPT

The prototype is a decision-making tool. The product is a delivery tool. They have different jobs and different architectures. The prototype's job is to make design decisions visible quickly. The product's job is to deliver those decisions reliably, at scale, with routing, performance, and maintainability. Moving from one to the other before the design decisions are stable produces technical debt. Moving from one to the other after they're stable produces clean architecture.

THE DECISION

Terrain's four screens were built and fully validated as HTML prototype files before a single Next.js page was written. The token system was committed. The CLAUDE.md was written. The project was moved to a standalone repo. Only then did app/page.tsx get created — with the prototype file as its direct reference. The first Next.js page matched the prototype exactly because all the decisions had already been made.

THE FRICTION

The standalone repo migration revealed a navigation assumption: HTML files linked to each other by relative path. Next.js routes by folder structure. The four screens needed new route architecture — /, /explore, /worker/blake, /path — that hadn't been planned in the prototype phase. Always sketch the route map before migrating, even if the pages are already designed.

THE TEACHING

The prototype earns its keep by making decisions cheap. Every round of iteration in HTML costs seconds. The same round in a Next.js component costs minutes plus a build cycle. Do all your deciding in the cheap environment. Ship in the expensive one. The moment the token system is committed and the CLAUDE.md is written, migrate. Not before.

"The prototype is done when you can answer every design question before the code asks it."

Terrain
Design System Reference

COLOUR TOKENS

--canvas-bg	#121314	Page background
--card-bg	#241E14	Card surface — warm earth
--card-border	rgba(245,240,235,0.15)	Card edge definition
--color-text-body	rgba(245,240,235,0.88)	Primary text — AAA
--color-text-muted	rgba(245,240,235,0.75)	Secondary text — minimum
--accent	#0047FF	Electric cobalt — one accent
--input-border-default	rgba(245,240,235,0.45)	Input at rest
--input-border-focus	#0047FF	Input active
--caret-color	#0047FF	Cursor

TYPOGRAPHY RULES

Minimum text size	**13px — no exceptions**
Minimum opacity	**0.75 — all text except placeholders**
Placeholder opacity	**0.45–0.55 — UX signal, not body text**
Label size	**0.85rem · letter-spacing: 0.08em**
Impact statements	**Georgia serif · display scale**
Body text	**System sans-serif · 16px base**
Labels/receipts	**IBM Plex Mono or system monospace**

DESIGN PRINCIPLES

— One emotion governs everything — hopeful.

— One accent colour (#0047FF) — used intentionally, never decoratively.

— Buttons fit their text — never fill containers.

— Information revealed progressively — never dumped.

— No social mechanics — no counts, feeds, or likes.

— The agent waits for conviction. Bring it.

SCREEN ARCHITECTURE

Screen 1	**/**	Landing — See the whole terrain.
Screen 2	**/explore**	Horizon Reveal — Six life patterns.
Screen 3	**/worker/blake**	Worker Story — Blake, electrician, Atlanta.
Screen 4	**/path**	Your Path — Three questions, one next step.

15

Automation layers

Hooks, routines, and the difference between asking and enforcing

CC4D Ch 7 + Ch 10

THE CONCEPT

CC4D Chapter 7 introduces automation hooks — PreToolUse and PostToolUse — as build constraints rather than guidelines. PreToolUse fires before Claude Code executes a command, allowing a block or transformation. PostToolUse fires after, enabling QA, formatting, or logging. Chapter 10 extends this into routines: named sequences of commands that run on a trigger. The distinction that matters: CLAUDE.md tells the agent what to prefer. Hooks tell the build system what to enforce. One is a request. The other is a rule.

THE DECISION

Terrain's enforcement layer uses three hooks: a PreToolUse hook that blocks direct hex value edits outside the token system, a PostToolUse hook that runs Prettier after every file write, and a completion hook that checks for hardcoded values before any commit. The three-hook recipe card in teaching/ documents the exact implementation. The hooks were added after Session 10 — after the token system was stable and the violation pattern was understood. Installing them earlier would have enforced rules that hadn't been decided yet.

THE FRICTION

The hardest part of hook design is knowing what not to enforce. The first version of the hex-block hook was too aggressive — it blocked intentional one-off overrides needed for the M3 reference layer. The fix: add an escape comment. Any line ending in `/* token-override */` passes the hook. The hook enforces the system without making exceptions impossible. Enforcement without escape hatches creates a build environment nobody wants to work in.

THE TEACHING

Teach hooks in this order: first show the violation they prevent, then show the hook that prevents it, then show the escape hatch. A hook without a demonstrated violation feels like bureaucracy. A hook that solves a real problem the student has already experienced feels like relief. The three-hook recipe — format on write, protect tokens, QA on complete — covers 80% of what any agentic design project needs. Start there.

"CLAUDE.md is a request. A hook is a rule. Know which one you need before you write it."

16

Programmatic video

When the design system extends into motion

TAD Ch 9

THE CONCEPT

TAD Chapter 9 covers Remotion and Hyperframes — frameworks for generating video programmatically from code rather than a timeline editor. The principle: if a design system can govern a UI, it can govern motion. The same tokens — colours, typography, spacing — that control a component control a frame. Video becomes a build output, not a separate production workflow. The designer who can code components can generate video. The gap between a website and a YouTube channel collapses to a JSON file and a render command.

THE DECISION

Terrain's YouTube channel uses Hyperframes running in the terrain-video project at ~/Documents/Cowork Playground/terrain-video. The project has its own CLAUDE.md, its own skill library under .agents/skills/hyperframes/, and a hyperframes.json that defines scenes, transitions, and timing. The Blake worker story was the first rendered video — three scenes, a narration track, and the Terrain token system applied to every frame. The video pipeline runs independently from the Next.js app but shares the same design language. Same cobalt. Same Georgia impact statements. Same hopeful.

THE FRICTION

The terrain-video project is easy to lose track of. It sits in the same Cowork Playground directory as terrain and portfolio-2026, but it has no Vercel deployment and no GitHub remote at time of writing. Two sessions were started in the wrong directory before the find . -name "CLAUDE.md" recovery pattern was established. The lesson: multi-project environments need a directory discipline that single-project work doesn't. Know which project you're in before you prompt.

THE TEACHING

Introduce programmatic video last, not first. Students need to understand the design loop, token systems, and MCP integrations before video production makes sense as an extension rather than a novelty. When it's introduced in the right order — after the design system is enforced, after the content pipeline is running — it lands as the natural conclusion: the same system that governs your product also governs your content. That realisation is worth waiting for.

"The same token system that governs the UI governs the frame. One design language. Two outputs."